

COMPUTACIÓN II – cuarta práctica:

(1) - Escribir un programa que multiplique las matrices A y B , y muestre por pantalla el resultado (matriz $C=B \cdot A$) de forma ordenada utilizando funciones.

$$A = \begin{pmatrix} 2 & 3 & 8 \\ 2 & 2 & 4 \\ 2 & 2 & 6 \end{pmatrix}; \quad B = \begin{pmatrix} 4 & 6 \\ 3 & 9 \\ 1 & 8 \end{pmatrix}$$

Para llevar a cabo el problema se recomienda proceder por partes:

- (a) Escribir un programa que lea las matrices A y B y las muestre por pantalla de forma ordenada.
- (b) A continuación, completar el programa para calcular el producto $C=A \cdot B$ mostrando el resultado de forma ordenada por pantalla.

$$C = A \cdot B = \begin{pmatrix} 25 & 103 \\ 18 & 62 \\ 20 & 78 \end{pmatrix}$$

- (c) Para finalizar, modificar el programa anterior de tal forma que el producto de las matrices y la impresión en pantalla de la matriz C se lleven a cabo mediante funciones.
- (*) El alumno que lo estime conveniente puede ir directamente al apartado (c).

Recordatorio notación matricial:

$$A = [A_{ik}] \ (n \times m), \text{ y } B = [B_{kj}] \ (m \times r) \rightarrow C = A \cdot B = [A_{ik}][B_{kj}] = [C_{ij}]$$

$$\text{con } C_{ij} = \sum_{k=1}^m A_{ik} B_{kj}, \ i = 1, 2, \dots, n, \ j = 1, 2, \dots, r \rightarrow (C_{11} = A_{11}B_{11} + A_{12}B_{21} + \dots + A_{1m}B_{m1})$$

En C++ : $A_{ij} \Rightarrow A[i][j]$

Recordemos que en C++ los elementos de un vector de dimensión n se numeran con los índices de 0 a $n-1$. Lo mismo sucede con las filas y columnas de una matriz de dimensión $n \times n$.

Apartado (1a):

```
#include <iostream>
using namespace std;

int main()
{
    const int fa=3, ca=3, fb=3, cb=2;
    int i, j,k;
    double A [fa][ca]={2,3,8,2,2,4,2,2,6};
    double B [fb][cb]={4,6,3,9,1,8};

    cout<<"Matriz A (3 filas x 3 columnas)";
    cout << endl;
    for (i=0; i<fa; i++){
        for (j=0;j<ca;j++){
            cout<<A[i][j]<<" ";
        }
        cout<<endl;
    }
    cout << endl;

    cout<<"Matriz B (3 filas x 2 columnas)";
    cout << endl;
    for (i=0; i<fb; i++){
        for (j=0;j<cb;j++){
            cout<<B[i][j]<<" ";
        }
        cout<<endl;
    }
    cout << endl;
    return 0;
}
```

Apartado (1b):

```
#include <iostream>
using namespace std;

int main()
{
    const int fa=3, ca=3, fb=3, cb=2, fc=fa, cc=cb;
    int i, j,k;
    double A[fa][ca]={2,3,8,2,2,4,2,2,6};
    double B[fb][cb]={4,6,3,9,1,8};
    double C[fc][cc];

    cout<<"Matriz A (3 filas x 3 columnas)";
    cout << endl;
    for (i=0; i<fa; i++){
        for (j=0;j<ca;j++){
            cout<<A[i][j]<<" ";
        }
        cout<<endl;
    }
    cout << endl;

    cout<<"Matriz B (3 filas x 2 columnas)";
    cout << endl;
```

```

    for (i=0; i<fb; i++){
        for (j=0; j<cb; j++){
            cout<<B[i][j]<<" ";
        }
        cout<<endl;
    }
    cout << endl;

// producto de matrices
double suma;
for (int i=0; i<fa; i++) { // fa=fc
    for (int j=0; j<cb; j++) { // cb=cc
        suma=0.0;
        for (int k=0; k<ca; k++) {
            suma=suma+A[i][k]*B[k][j];
        }
        C[i][j]=suma;
    }
}
// muestra C
cout<<"Matriz producto, C (3 filas x 2 columnas):" << endl;
for (int i=0; i<fc; i++) {
    for (int j=0; j<cc; j++) {
        cout << C[i][j] << " ";
    }
    cout << endl;
}
return 0;
}

```

Apartado (1c):

```

#include <iostream>
using namespace std;
const int fa=3, ca=3, fb=3, cb=2, fc=fa, cc=cb;
void multmat(double A[fa][ca], double B[fb][cb], double C[fc][cc]);
void muestramatriz(double C[fc][cc]);

int main()
{
    double A[fa][ca]={2,3,8,2,2,4,2,2,6};
    double B[fb][cb]={4,6,3,9,1,8};
    double C[fc][cc];
    multmat(A,B,C);
    muestramatriz(C);
    return 0;
}

void multmat(double A[fa][ca], double B[fb][cb], double C[fc][cc])
{
    double suma;
    for (int i=0; i<fa; i++) {
        for (int j=0; j<cb; j++) {
            suma=0.0;
            for (int k=0; k<ca; k++) {
                suma=suma+A[i][k]*B[k][j];
            }
            C[i][j]=suma;
        }
    }
}

```

```
void muestramatriz(double C[fc][cc])
{
    cout<<"Matriz producto:"<<endl;
    for (int i=0; i<fc; i++) {
        for (int j=0; j<cc; j++) {
            cout << C[i][j] << " ";
        }
        cout << endl;
    }
}
```

```

#include <iostream>
using namespace std;
const int ca=3, cb=2, cc=cb;
int fa=3, fb=3, fc=fa;
void multmat(double A[][ca], double B[][cb], double C[][cc], int fa);
void muestramatriz(double C[][cc], int fc);
// En este caso solo se definen como constantes la dimension de las columnas
// Este es el requisito mínimo cuando se trabaja con arrays no dinamicos
// Si se necesitan las filas en la funcion hay que pasarlas como argumentos
int main()
{
double A[][ca]={2,3,8,2,2,4,2,2,6};
double B[][cb]={4,6,3,9,1,8};
double C[fb][cc];
multmat(A,B,C,fa);
muestramatriz(C,fc);
return 0;
}

void multmat(double A[][ca], double B[][cb], double C[][cc], int fa)
{
double suma;
for (int i=0; i<fa; i++) {
    for (int j=0; j<cb; j++) {
        suma=0.0;
        for (int k=0; k<ca; k++) {
            suma=suma+A[i][k]*B[k][j];
        }
        C[i][j]=suma;
    }
}
}

void muestramatriz(double C[][cc], int fc)
{
for (int i=0; i<fc; i++) {
    for (int j=0; j<cc; j++) {
        cout << C[i][j] << " ";
    }
    cout << endl;
}
}

```

(2)- Escribir una función que reciba como parámetro una matriz cuadrada de orden n y calcule la transpuesta de la matriz almacenando el resultado en la propia matriz.

```
#include <iostream>
using namespace std;
const int n=3;
void transpuesta(double a[n][n], int n);
void muestramatriz(double a[n][n], int n);

int main()
{
double a[n][n]={1,2,3,4,5,6,7,8,9};
cout << "Matriz original" << endl;
muestramatriz(a,n);
cout << "Matriz transpuesta" << endl;
transpuesta(a,n);
muestramatriz(a,n);
cout << endl;
return 0;
}

void transpuesta(double a[n][n], int n)
{
double aux;

for (int i = 0; i < n; i++)
for (int j = 0; j < i; j++)
{
aux = a[i][j];
a[i][j] = a[j][i];
a[j][i] = aux;
}
}

void muestramatriz(double a[n][n], int n)
{
for (int i=0; i<n; i++) {
for (int j=0; j<n; j++) {
cout << a[i][j] << " ";
}
cout << endl;
}
}
```